

PRODUCT REQUIREMENTS DOCUMENT

Behavioral Fraud-Risk Triage

An AI-assisted review queue for the fraud operations desk

Product area	Fraud & Financial Crime — Operations Tooling
Document type	Feature PRD (v1 — MVP scope)
Author	Reza Rahid Sami, Product Manager (candidate portfolio)
Status	Draft for review
Framing	Internal risk-operations tool at a retail bank

This PRD reframes a working prototype (an agentic fraud-risk reviewer built on ~2,500 real bank transactions and independently evaluated against a labeled dataset) as the product a fraud-operations team would actually ship. Technical results cited throughout come from that build.

Contents

1. Summary

Fraud analysts on the review desk today work a queue of transactions that a legacy rules engine has flagged. The queue is noisy: most flagged items are legitimate, each still has to be opened and read, and the rules give a reason like “*amount > \$X*” that rarely reflects why *this* account’s behavior is actually unusual. Analysts burn time triaging obvious false alarms, and genuinely risky patterns that no single rule catches slip through.

This feature adds an **AI-assisted triage layer** in front of the analyst queue. For every transaction it computes **account-relative behavioral signals** (how unusual an amount is for that specific account, device/location/IP changes, share of balance, login attempts, timing), hands those signals to a language-model agent, and gets back a **continuous 0–100 risk score with a plain-English reason**. Analysts work the queue sorted by score, read the reason before opening anything, and the risk team tunes one threshold to trade off missed fraud against review volume — without re-running the model.

WHY IT MATTERS

On an independent labeled evaluation, the scoring approach reached **91.5% recall at 83.2% precision** (F1 87.1%) — up from 46.5% recall in the first version. In queue terms: catch roughly 9 in 10 real fraud cases while keeping about 5 of every 6 flags actionable. See Section 8 for the honest reading of these numbers.

2. Problem & context

2.1 Who feels the pain

- **Fraud analysts (primary):** work the review queue transaction by transaction. Their time is the scarcest resource on the desk; every false alarm they open is capacity not spent on a real case.
- **Fraud-ops leads (secondary):** own the tradeoff between fraud losses and analyst workload, and answer for both. Today they can only tune rigid rule thresholds, one rule at a time.
- **Customers (indirectly):** a false decline or an unnecessary hold on a legitimate transaction is a real customer-experience cost, not a free action.

2.2 What today's workflow looks like

1. A rules engine flags transactions on fixed conditions (amount over a threshold, foreign location, etc.).
2. Flagged items land in an undifferentiated queue — no ordering by how risky they actually are.
3. An analyst opens each one, pieces together context from raw fields, and decides: clear, hold, or escalate.
4. The flag “reason” is the rule that fired, which often doesn’t map to why this account’s behavior is unusual.

2.3 Where it breaks

- **Flat thresholds ignore the account.** A \$500 charge is unremarkable on one account and a red flag on another. A single dollar cutoff treats them identically, producing false alarms on high-spenders and misses on low-spenders.
- **No prioritization.** A binary flag can’t tell an analyst which of 200 queued items to open first.
- **Weak explanations.** “Rule 14 fired” forces the analyst to reconstruct context from scratch every time.
- **Combinations slip through.** New device *and* new location *and* an unusually large amount together is far more suspicious than any one alone — but rule engines evaluate conditions in isolation.

3. Goals & non-goals

3.1 Goals

- **Cut time analysts spend on false alarms** by surfacing a score and a reason before they open anything.
- **Prioritize the queue** so the riskiest transactions are worked first.
- **Give every flag a human-readable, evidence-grounded reason.**
- **Put the fraud-vs-workload tradeoff in the risk team's hands** as a single tunable threshold over a stored score.
- **Avoid demographic bias by construction** — age and occupation never enter the scoring logic.

3.2 Non-goals (explicitly out of scope for v1)

- **Auto-actioning.** The agent scores and explains; it never blocks, holds, or declines on its own. A human makes every decision.
- **Replacing the rules engine.** v1 sits alongside existing controls as a triage layer, not a rip-and-replace.
- **Real-time authorization scoring.** v1 targets the post-flag review queue, not the sub-second authorization path.
- **Model-driven case adjudication or SAR filing.** Downstream case management is unchanged.

4. User & proposed workflow

4.1 Primary persona

PERSONA — MAYA, FRAUD ANALYST

Works a shift-based review queue; measured on cases cleared and fraud caught. Comfortable with data but not a data scientist. Wants to open fewer junk cases and trust that when something is scored high, there's a real reason behind it. Frustrated today by a queue with no ordering and reasons that don't explain much.

4.2 Target workflow with this feature

5. Transactions flow in and the system computes behavioral signals per transaction, each relative to that account's own history.
6. The agent assigns a 0–100 risk score and, for anything meaningful, a short reason.
7. The queue is sorted by score, highest first. Maya sees score, amount, channel, and reason at a glance.
8. She works top-down. High scores get attention first; low scores can be batch-cleared with confidence.
9. She can open any item to see the underlying signals, ask the agent to explain a flag in more depth, or query the batch in plain language ("which accounts had the most flags?").
10. Fraud-ops leads adjust the flagging threshold centrally as loss experience and staffing change — instantly, over already-scored transactions.

5. Solution & requirements

The system separates “compute the evidence” from “make the judgment call” on purpose: a feature-engineering layer produces signals but decides nothing, and a single agent layer is the only place a risk score is ever assigned. That boundary keeps scoring auditable and prevents ad-hoc logic from leaking across the codebase.

5.1 Behavioral signals (feature layer)

Every signal is measured against the account's own baseline, not a flat rule:

Signal	What it captures	Why it matters
Amount z-score	How unusual the amount is for this account	Account-relative, so it adapts to each customer's normal
Duration z-score	How unusual the transaction's duration is for this account	Off-pattern session behavior
% of balance	Share of account balance this one transaction moves	Near-total drains are a classic takeover pattern
Hours since previous	Real gap since the account's last transaction	Rapid back-to-back activity is a risk signal
Device / location / IP / channel changed	Change vs. the account's prior transaction	Combinations are far stronger than any single change
Login attempts	Failed attempts before this transaction	Can indicate credential stuffing / brute force
Account transaction count	How much history backs this account	Down-weights "changed" signals for thin-history accounts

DESIGN DECISION — COLD START / SHRINKAGE

With only a handful of transactions per account, a z-score from an account's own data is noise — one ordinary purchase can look extreme by chance. The system blends each account's own mean/variance with the population baseline, weighted by how much history it actually has (shrinkage estimation). Thin-history accounts lean on the population; established accounts lean on themselves. This is what stops "new device" from firing on an account seen only twice.

5.2 Risk scoring (agent layer)

- **Continuous 0–100 score, not binary.** The model expresses graded confidence; the flag is derived in code from a threshold, so the score is preserved and the cutoff is tunable after the fact.
- **Holistic weighting.** The agent is instructed to combine signals rather than fire on any one — a lone device change nudges the score slightly; device + location + unusual amount together move it a lot.
- **Plain-English reason on every meaningful score,** written for an analyst, not a log file.
- **No demographics.** Age and occupation are withheld from scoring entirely (shown to the human only as context).

5.3 Analyst surface (review UI)

- Queue sorted by risk score, with score, amount, channel, location, and reason visible per row.
- A threshold control that re-derives which items count as flagged, live, without re-scoring.
- A detail view exposing the underlying signals behind any score.
- On-demand "explain this flag" and a plain-language assistant for querying the batch.

5.4 Trust & safety requirements (must-have for v1)

- **Schema validation.** Every model output is checked for shape and type before it can touch the database; malformed results fall back safely instead of corrupting the queue.
- **Grounding check.** If the model cites "unusually large amount" but the amount z-score doesn't support it, the mismatch is surfaced rather than silently trusted.
- **No unexplained high scores.** A meaningfully risky score without a reason is rejected as malformed.

5.5 Scope: MVP vs. later

Capability	MVP (v1)	Later
Behavioral scoring + reasons on the review queue	Yes	—
Continuous score with tunable threshold	Yes	—
Analyst detail view + explain + assistant	Yes	—
Validation + grounding safeguards	Yes	—
Analyst feedback capture (agree / disagree on a flag)	—	v2
Threshold auto-tuning from loss data	—	v2
Real-time authorization-path scoring	—	Future
Feedback loop retraining / prompt refinement	—	Future

6. Success metrics

6.1 Product metrics (the real goal)

Metric	Definition	Target direction
Analyst time per cleared case	Avg. handling time on the review queue	Down
False-alarm open rate	Share of opened cases that were legitimate	Down
Fraud caught in review	Confirmed fraud identified from the queue	Up / hold
Queue throughput	Cases cleared per analyst-hour	Up
Customer-impact rate	Legit transactions held or declined in error	Down

These are what the feature is for. A model that scores well but doesn't move analyst time or false-alarm rate hasn't shipped value — so the model metrics below are treated as leading indicators, not the goal.

6.2 Model metrics (leading indicators)

- **Recall** — share of real fraud the system flags (the cost of a miss is high).
- **Precision** — share of flags that are real fraud (the cost of a false alarm is analyst time).
- **F1** — balance of the two, tracked as the threshold moves.
- **Grounding-warning rate** — how often the model's stated reason isn't supported by its own data (a reasoning-quality signal).

7. Key tradeoffs & decisions

7.1 Precision vs. recall is a business choice, not a model constant

Because scoring is continuous, one threshold sets the whole tradeoff. Lower it to catch more fraud at the cost of more false alarms; raise it to protect analyst time at the cost of more misses. This is exactly the lever fraud-ops leads should own, and it moves without re-running the agent. Measured threshold behavior:

Threshold	Effect	Who chooses
Lower (e.g. 40–50)	Higher recall, more false alarms, heavier queue	Fraud-ops lead
Default (60)	Balanced starting point	Default
Higher (e.g. 70–80)	Fewer false alarms, more missed fraud	Fraud-ops lead

7.2 Continuous score vs. binary flag

Moving from a model-decided yes/no to a continuous score was the single biggest quality gain in development (recall jumped substantially once the model could express graded confidence instead of being forced into a premature yes/no). It also enables prioritization and post-hoc threshold tuning. Decision: continuous scoring is a v1 requirement, not an enhancement.

7.3 Explainability vs. raw accuracy

Requiring a grounded reason on every meaningful score is a deliberate constraint on the model. In testing it improved judgment quality, not just readability — forcing the model to cite only what the data supports made its decisions better, not merely better-worded. Decision: keep the constraint even if a freer model scored marginally higher, because an unexplained flag isn't actionable on a review desk.

7.4 Fairness by construction

Demographic fields are withheld from scoring entirely rather than “used carefully.” This forgoes any predictive signal age or occupation might carry, in exchange for removing a well-documented bias vector and a compliance risk. For a regulated bank, that trade is worth making explicitly.

8. Evidence & honest reading of results

The scoring approach was evaluated against an independent labeled dataset (a balanced sample of 200 fraud + 400 legitimate transactions) at the default threshold of 60:

Metric	Result	In plain terms
Precision	83.2%	~5 of every 6 flags are real fraud
Recall	91.5%	catches ~9 of every 10 real fraud cases
F1	87.1%	balance of the two
Accuracy	91.0%	vs. a ~66.7% floor for never-flagging

8.1 The improvement was measured, not assumed

Version	Precision	Recall	F1	Accuracy
Initial (binary, no grounding check)	74.4%	46.5%	57.2%	76.8%
+ validation & grounding check	76.6%	54.0%	63.3%	79.2%
+ continuous risk scoring	83.2%	91.5%	87.1%	91.0%

STATED LIMITATION (KEPT IN, ON PURPOSE)

In this evaluation dataset, two reputation features correlate with the fraud label at ~0.87 — an unusually

strong, near-deterministic signal for synthetic data. These results therefore likely **overstate** how well the approach would generalize to messier real-world signals, where no single feature is that predictive. A production rollout must re-baseline against the bank's own labeled outcomes before any of these numbers are treated as expected performance.

9. Rollout plan

11. **Shadow mode.** Score the live queue without changing analyst workflow; compare scores and reasons against analyst outcomes to baseline precision/recall on the bank's own data.
12. **Pilot with one analyst pod.** Surface the score, ordering, and reasons to a small team. Measure time-per-case and false-alarm open rate against a control group.
13. **Threshold calibration.** Fraud-ops leads set the launch threshold from pilot loss/workload data rather than the default.
14. **Phased rollout.** Expand pod by pod, watching the product metrics in Section 6, with the rules engine still running underneath.
15. **Add the feedback loop (v2).** Capture analyst agree/disagree to drive threshold auto-tuning and prompt refinement.

10. Risks & mitigations

Risk	Mitigation
Model hallucinates a reason unsupported by the data	Schema validation + grounding check reject/surface unsupported reasons; no unexplained high scores reach the queue
Results don't generalize from eval data to production	Shadow-mode re-baselining on the bank's own labeled outcomes before launch (Section 9)
Over-reliance / automation bias by analysts	Human decides every case; agent scores and explains only — no auto-actioning in v1
Bias against protected groups	Demographic fields withheld from scoring by construction
Cold-start noise on thin-history accounts	Shrinkage estimation down-weights an account's own thin history toward the population baseline
Throughput / cost at production scale	Batched scoring, rate-limit backoff, and threshold tuning that needs no re-scoring keep the queue responsive

11. Open questions

- What's the bank's true cost ratio of a missed fraud vs. an unnecessary review? This sets the launch threshold.
- Should the score augment the existing rules queue, or eventually replace specific rules once shadow-mode data justifies it?
- What latency budget does the review path allow, and does any of this need to move toward the authorization path later?
- What's the model-governance and audit requirement for an LLM in a regulated fraud-decisioning workflow?

Prepared as a portfolio PRD. Metrics and design decisions are drawn from a working prototype build; production numbers must be re-established on the bank's own data.